



Hashed and Hierarchical Timing Wheels: Data Structures for the Efficient Implementation of a Timer Facility

NOVEMBER 23, 2015 JULY 27, 2017

Hashed and Hierarchical Timing Wheels: Data Structures for the Efficient Implementation of a Timer Facility (<http://www.cs.columbia.edu/~nahum/w6998/papers/sosp87-timing-wheels.pdf>). – Varghese & Lauck 1987

Yashiro Matsuda recently wrote a blog post describing Apache Kafka's use of Hierarchical Timing Wheels to keep track of large numbers of outstanding requests (<http://www.confluent.io/blog/apache-kafka-purgatory-hierarchical-timing-wheels>). In the Kafka use case, each request lives in a 'purgatory' data structure and is associated with a timeout timer and a map of watcher lists for event driven processing. Efficiently keeping track of expiry timers is a common problem. The principle can apply to any system keeping track of outstanding requests or messages with an expiry time.

Today's choice is the 1987 paper by Varghese and Lauck in which they study a number of different approaches for the efficient management of timers, and introduce the concept of hierarchical timing wheels as used by Kafka. They model timers as composed of two user-facing operations, *start* and *stop*, and two internal operations: *per-tick bookkeeping* and *expiry processing*.

- Start timer is called by clients specifying a timer duration and a callback. In the author's model the client also passes in a *request id* to distinguish the timer, though nowadays we would be more inclined to return a *timer-id* in response to a start-timer request.
- Stop timer takes a request (timer) id and finds and stops (removes) the associated timer.
- Per-tick bookkeeping happens on every 'tick' of the timer clock. If the unit of granularity for setting timers is T units of time (e.g. 1 second), then per-tick bookkeeping will happen every T units of time. It checks whether any outstanding timers have expired, and if so it removes them and invokes *expiry processing*.
- Expiry processing is responsible for invoking the user-supplied callback (or other user requested action, depending on your model).

Different data structures and algorithms have differing complexities in the costs of performing these operations (for example, is starting a timer a constant time operation, or does it depend on the number of existing timers or perhaps even some other variable?). We get seven different schemes for managing timers, with the guidance that "for a general timer module... that is expected to work well in a variety of environments, we recommend scheme 6 or 7." Scheme 6 is 'hashed timing wheels' and scheme 7 is 'hierarchical timing wheels'.

Let's take a look at the schemes...

1. Unordered Timer List

Keep an unordered list of timers and track the remaining time for each. On start, simply add the new timer to the list. Per-tick bookkeeping must traverse the full list and decrement the remaining time for each timer on every tick. If a timer reaches zero, it is removed from the list and expiry processing is invoked. Starting a timer is therefore $O(1)$, stopping a timer is $O(1)$, and per-tick bookkeeping is $O(n)$ where n is the number of outstanding timers.

2. Ordered Timer List

Keep a list of timers as in scheme 1, but record the absolute expiry time (not remaining time) and keep the timer list ordered by this expiry time (with the timers closest to expiry at the head of the list). On each tick compare the expiry time of the timer at the head of the list with the current wall clock and remove & expire the timer if its expiry time is $\leq$ the current time. Keep doing this until the head of the list contains a timer with an expiry time in the future.

Starting a timer is now $O(n)$ due to searching the correct place in the list to insert it, but per-tick bookkeeping is $O(1)$.

3. Timer Trees

For large n , we can improve on scheme 2 by keeping timers in a tree-based data structure. This means we can insert (start) timers in $O(\log(n))$ vs $O(n)$ for the ordered list.

4. Simple Timing Wheels

The basic timing wheel approach is applicable when all timers have a maximum period of no more than $MaxInterval$, and we can construct a circular buffer with $MaxInterval$ slots (each representing one tick). The current time is represented by an index into the buffer. To insert a timer that expires $j \leq MaxInterval$ time units in the future, we move j slots around the ring and add the timer to a list of timers held in that slot. Every tick, the current time index moves one slot around the ring and does expiry processing on all timers held in the new slot. Start, stop, and per-tick bookkeeping are all $O(1)$.

5. Hashing Wheel with Ordered Timer Lists

If $MaxInterval$ is comparatively large (e.g. 32-bit timers), simple timing wheels can use a lot of memory. Instead of using one slot per time unit, we could use a form of hashing instead. Construct a circular buffer with a fixed number of slots – a power of 2 for efficiency and have the current time

index advance one position in the ring on a tick as before. To insert a timer that expires j time units in the future, compute a slot delta $s = j \% \text{num-buckets}$. Insert the timer s slots around the ring with its time of expiry. Since there may be many timers in any given slot, we maintain an ordered list of timers for each slot.

Per-tick bookkeeping advances the current time index and processes the list of timers found there as in scheme 2. The worst case latency for inserting a timer is $O(n)$, but the average is $O(1)$. Per-tick bookkeeping is $O(1)$.

6. Hashing Wheel with Unordered Timer Lists

This is a variant on scheme 5 where instead of storing absolute time of expiry we store a count of how many times around the ring each timer is in the future. To insert a timer that expires j time units in the future, compute a counter value $c = j / \text{num-buckets}$ and a slot delta $s = j \% \text{num-buckets}$. Insert the timer s slots around the ring with its counter value c . Keep the timers in an unordered list in each slot.

Starting a timer now has both worst and average case $O(1)$, and per-tick bookkeeping is worst case $O(n)$, but $O(1)$ on average.

7. Hierarchical Timing Wheels

Another way to deal with the memory issues caused by the simple timing wheel approach is to use multiple timing wheels in a hierarchy. Suppose we want to store timers with second granularity, that can be set for up to 100 days in the future. We might construct four wheels:

- A *days* wheel with 100 slots
- An *hours* wheel with 24 slots
- A *minutes* wheel with 60 slots
- A *seconds* wheel with 60 slots

This is a total of 244 slots to address a total of 8.64 million possible timer values. Every time we make one complete revolution in a wheel, we advance the next biggest wheel by one slot (the paper describes a slight variation with minute, hour, and day ticking clocks, but the effect is the same). For example, when the seconds wheel has rotated back to index '0' we move the index pointer in the minutes wheel round by one position. We then take all the timers in that slot on the minutes wheel (which are now due to expire within the next 60 seconds) and insert them into their correct positions in the seconds wheel. Expiry time processing in the seconds wheel works exactly as described in scheme 4 (it's just a simple timer wheel that happens to get replenished on every revolution).

To insert a timer, find the first wheel (from largest units to smallest) for which the timer should expire 1 or more wheel-units into the future. For example, a timer due to expire 11 hours, 15 minutes and 15 seconds into the future would be inserted at slot 'current-index + 11' in the hours wheel, storing the remainder of 15 minutes and 15 seconds with the timer. After the hours wheel has advanced by 11 positions, this timer will be removed from that wheel and inserted at 'current index + 15' slots round in the minutes wheel, storing the remainder of 15 seconds. When the minutes wheel has

subsequently advanced by 15 positions, this timer will be removed from the wheel and placed in the seconds wheel at 'current index + 15' slots round in the seconds wheel. 15 seconds later, the timer will expire!

Note: the paper uses the seconds, minutes, hours, days example and it certainly makes it easy to follow, but if you just get given timers as e.g. t seconds in the future for up to a 32 bit timer value, then it would be more efficient to simply divide this into four wheels with 2^8 slots in each, or similar (which makes it very efficient to determine which wheel an entry should go in).

Choosing between schemes 6 and 7...

Whether scheme 6 or 7 is better in any given situation depends on a number of parameters:

- n , the number of timers
- M , the total number of slots available
- m , the number of levels (for the hierarchical approach)
- T , the average timer interval
- $\text{cost}_{\text{index}}$ the cost of hashing and indexing under scheme 6 for one entry
- $\text{cost}_{\text{migrate}}$ the cost of moving timer entries to the next wheel under scheme 7.

For scheme 6 the cost is approximately $n \cdot \text{cost}_{\text{index}} / M$, and for scheme 7, $nm \cdot \text{cost}_{\text{migrate}} / T$.

Since $\text{cost}_{\text{index}}$ and $\text{cost}_{\text{migrate}}$ will not be drastically different, for small values of T and large values of M , Scheme 6 can be better than Scheme 7 for both START-TIMER and PER-TICK-BOOKKEEPING. However, for large values of T and small values of M , Scheme 7 will have a better average cost (latency) for PER-TICK-BOOKKEEPING but a greater cost for START-TIMER.

ALGORITHMS AND DATA STRUCTURES

4 thoughts on “Hashed and Hierarchical Timing Wheels: Data Structures for the Efficient Implementation of a Timer Facility”

1. Pingback: [RethinkDB и Pi Zero — Episode 0069 « DevZen Podcast](#)

2. Pingback: [A Year in Papers | the morning paper](#)

3. [kzzircuit](#)

SAYS:

FEBRUARY 9, 2018 AT 5:00 AM

Reading this article was pure bliss because it is concise and provides the concepts with such an ease. Thanks!

4. [Joel](#)

SAYS:

JULY 30, 2019 AT 8:06 AM

Shouldn't insertion into a sorted timer list be $O(\log(n))$ because you can easily find the insertion point with a binary search?

Comments are closed.

BLOG AT WORDPRESS.COM.